

FORKCAST

FORKCAST (*student proposal*)

Assessment: ✓ Fit · **Category:** Social Media · **Assessed:** 2026-09-22 ·
Status: awaiting instructor approval

Proposal (as submitted): *Forkcast* helps users and friend groups decide where to eat. A **Dining Decision Bracket** lets everyone swipe through restaurant options to narrow choices and break ties; the app compares everyone's picks into a final recommendation. As users visit restaurants, the app learns their **Food Taste DNA**; a **Community Hub** lets them find people with similar tastes (by preferences or Taste DNA), join cuisine communities, chat, and share reviews/discoveries. A **Cuisine Tracker** logs cuisines/restaurants tried and awards **milestone badges** shown off on the profile. Named load-bearing C++: the **Bracket Simulation Engine** — a live single-elimination state machine seeded from an integer + a restaurant-ID vector, swipe-driven, adding restaurants each swipe until a threshold/pick resolves; the **Food Taste DNA** performant service — a weighted vector/map (one dimension per cuisine tag) updated per dine-in with a **decayed running average** scaled by rating, with a weightage hierarchy (post-dine rating > repeat visits > dining-in-after-swiping > bookmarking > joining a hub > swiping); the **Community Hub** — **TCP** for chat text (server-relayed, buffered against overflow) and **UDP with a sliding window** for video/audio/image (chunked, missing-chunk detection + retransmit), event-driven high-throughput; and the **Cuisine Tracker** — **Firestore** (Users, Restaurants, Reviews, Leaderboards) with matching C++ data structures, a **Linux-VM backend script pulling restaurants from the Google Places API** and tagging cuisine/area into Firestore, C++-core review validation submitted through services to the backend, and backend cuisine-count + milestone-badge tracking. Team: **6 CS + 1 Design + 0 Art**.

Note (category): Forkcast's core is **social** — cuisine communities, live chat + media, connecting people by Taste DNA, shared reviews, and show-off badge profiles — so it is assessed as **Social Media**; the bracket engine and Taste DNA are the decision/recommendation systems that feed the social network. (Runners-

up considered: Visualization & Analytics for the Taste-DNA modelling, and a gamified-app lens for the badges.)

Note (module completeness): the design already leans on **Firestore + a Linux VM backend + Google Places API** — this is exactly the module's model (external DB mandatory; a Linux server set up by M2, full use T2). *Platform:* the swipe-first, on-the-go experience is a **mobile app** (Native Android, Pixel 9a). *Scope caution:* the **UDP sliding-window media streaming** (video/audio) in chat is genuinely hard — treat **text chat as the floor** and media as a stretch (see scope note).

References:

- [Beli — Wikipedia](#) (a social app to track, rank and share restaurant experiences with friends, whose algorithm computes a "Match Score" to connect users with similar tastes and recommend places — the restaurant-tracking + taste-matching + social precedent, almost feature-for-feature)
- [Untappd — Wikipedia](#) (a social check-in app where users log what they consume, earn **milestone badges**, and share to a community feed — the Cuisine-Tracker + badges + community precedent)
- [Tinder — Wikipedia](#) (the origin of the swipe-through-a-stack-of-cards decision mechanic — swipe to like/dislike to narrow a set — the Dining Decision Bracket's swipe-UX precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	FORKCAST
External Database	✓
Authentication »	●
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	✓

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — C++ engine/services via JNI, OpenGL ES for the Taste-DNA visualization in a GLSurfaceView, Kotlin app shell; device: Pixel 9a) — a swipe-first, on-the-go social app; some screens (feed, profile, hub) can be pure Native UI. The Native PC build is the dev/tools vehicle (admin/moderation + restaurant/badge tooling). **Extended:** Firebase Firestore + a Linux VM backend (set up by M2, full use T2) — the Google Places ingest script, review persistence, cuisine-count/badge tracking and the chat relay.

Front end: A 2D social app — the swipe-bracket cards, the Food Taste DNA visualization, the Community Hub (communities, chat, media), review forms, and the profile with badges/leaderboards. **Languages:** Native — Kotlin Native Android UI + the C++ engine/services via JNI (Android Studio); OpenGL ES for the custom Taste-DNA charts in a GLSurfaceView; feed/profile/hub screens can be pure Native UI. Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: C++ core + services — the Bracket single-elimination state machine, the Food Taste DNA weighted-vector service (decayed running average), the chat relay (TCP text, buffered) + UDP sliding-window media, and review validation — plus a Linux VM backend (Google Places ingest, badge/cuisine tracking) and Firebase Firestore (Users/Restaurants/Reviews/Leaderboards) with matching C++ data structures. **Languages:** C++ (engine/services); backend script on the Linux VM; Firebase on the Extended tier.

Backend scope: Client + services now, cloud + server (T2) — Firebase live from M1 for identity/data; the Linux VM chat relay + Places ingest set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* three load-bearing C++ pieces — the **Bracket state machine** (seeded, swipe-driven single-elimination resolving to a recommendation), the **Food Taste DNA** service (per-cuisine weighted vector, decayed running average scaled by the weightage hierarchy), and the **chat/networking** event loop (high-throughput, buffered).
- *Networking: core* — TCP server-relayed chat with overflow buffers, and UDP sliding-window media (chunk detection + retransmit). Text chat is the floor; the custom reliable-UDP media is the stretch. Application Communication ✓ covers client↔Firebase↔backend.
- *Data-driven:* users, restaurants (Places-ingested, cuisine/area-tagged), reviews, cuisine tags/weights, badge milestones and leaderboards are all data — matching C++ structs and Firestore models; a new cuisine tag or badge tier is a data change.
- *2D:* the swipe-bracket cards, the **Food Taste DNA visualization** (custom-rendered charts, no chart libs) and the profile/badges satisfy the 2D gate. *3D:* not needed.
- *Front-end + Back-end:* a genuine split — mobile client + C++ services + Linux-VM backend + Firestore; the Places ingest and review-submission flows are **Application Communication** ✓.

- *Custom tool*: an **admin / moderation + restaurant-and-badge management** dashboard (review moderation, cuisine-tag/badge-milestone editing, the Places-ingest control) — the "tools development" the proposal calls for to keep the system stable.
- *External DB*: central (Firestore — users/restaurants/reviews/leaderboards).
Auth » / Analytics »: accounts (Firebase identity, foundational here) and engagement analytics are natural but module-deferred to T2 (●).

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Social Media pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the shell + the **Bracket** state machine + swipe input.

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a C++ loop hosting the single-elimination Bracket state machine (seeded integer + restaurant-ID vector, swipe-driven resolution) and swipe input; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the Taste DNA (M1G9) and networking (M1G4) build on it.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** C++ structs (User, Restaurant, Review, cuisine tag/weight, badge milestone) mirrored to Firestore models, in hot-reloadable JSON where local — a new cuisine tag or badge tier is a data change; the shared model the app + DB both use.
- **Prerequisites:** G1 (loop & entity hooks).
- **Feeds into:** M1G8 (Firebase), M1G9 (Taste DNA), M2G2 (admin tool).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui/overlay inspectors (bracket state, Taste-DNA weights, chat buffer occupancy), Tracy profiling of the chat relay + Taste-DNA updates under load, Catch2 tests (bracket resolution, decayed-average correctness, data loading), on a GitHub Actions Windows+Linux matrix.
- **Prerequisites:** G1 (loop to instrument).

- **Feeds into:** M2G3 (CI hardening); the network stability tooling.
- **G4 Real-time Networking Foundation** — the Community Hub chat transport (TCP relay + presence).
Tech & dependencies
 - **Tech: Web build:** Winsock2/BSD sockets (or ENet/SDL_net), a TCP chat relay (client→server→client) with presence + a message protocol and overflow buffers, serialization; the base the UDP media layer extends. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** G1 (shell & loop).
 - **Feeds into:** M2G9 (Chat II — rooms/presence/history); the Community Hub.
- **G8 Firebase Integration — Identity & Data** — users, restaurants, reviews, leaderboards + the Places ingest.

Tech & dependencies

- **Tech:** Firebase Auth + Firestore (Users/Restaurants/Reviews/Leaderboards), REST from the C++ core; a Linux-VM backend script pulling restaurants from the Google Places API, tagging cuisine/area, writing to Firestore; C++-core review validation submitted to the backend.
- **Prerequisites:** G2 (matching data model).
- **Feeds into:** M2G7 (Firebase auth/push/analytics); the Cuisine Tracker/badges; the social graph.
- **G9 Matching / Ranking Engine — Foundation (Food Taste DNA)** — the weighted-vector affinity service.

Tech & dependencies

- **Tech:** A per-cuisine-tag weighted vector/map, each dine-in updating a tag via a decayed running average scaled by the weightage hierarchy (post-dine rating > repeat visits > dine-in > bookmark > join hub > swipe); weighted scoring for the bracket's group aggregation (custom C++).
- **Prerequisites:** G2 (user/content model).
- **Feeds into:** M2G8 (Ranking II — recommendations + similar-taste matching); the bracket's final recommendation.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — on-device perf + the Linux-VM relay/backend seam.

Tech & dependencies

- **Tech: Web build:** Profiling + optimization of the chat relay + Taste-DNA service (threading, memory pools), sanitizers, and on-device validation (Pixel 9a); the Linux VM (chat relay + Places ingest) + Firebase set up (account + software + basic connectivity test) by M2.
- **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The mobile deliverable + a stable, scalable hub.

- **G2 ★ Content Editor — Core (Admin / Moderation & Restaurant-Badge Tool)** — the stability/admin tooling.

Tech & dependencies

- **Tech:** Dear ImGui (or a web admin UI): moderate reviews/chat, edit cuisine tags + badge milestones, control the Places-ingest, and inspect leaderboards, with round-trip JSON/Firestore and undo/redo.
- **Prerequisites:** G2 (data model), M1G8 (Firestore).
- **Feeds into:** Safe deployment; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android builds, tests (bracket resolution, decayed-average, chat under simulated latency/loss, data loading) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.

- **G8 Ranking / Matching Engine II — Recommendations & Similar-Taste Discovery** — the Taste DNA matured.

Tech & dependencies

- **Tech:** Scoring models over the Taste-DNA vectors (weighted features, cosine similarity), candidate generation for restaurant recommendations, and user-to-user matching (find people with similar Taste DNA) with evaluation metrics.
 - **Prerequisites:** M1G9 (Taste DNA foundation).
 - **Feeds into:** The Community Hub's "similar tastes" discovery; the bracket recommendations.
- **G9 Chat II — Rooms, Presence, Typing, History** — the Community Hub matured.

Tech & dependencies

- **Tech:** Cuisine-community rooms, presence + typing indicators, message history (Firestore), and (stretch) the UDP sliding-window media path (chunked send + retransmit) layered on M1's transport.
 - **Prerequisites:** M1G4 (networking foundation).
 - **Feeds into:** The Community Hub showcase; moderation pipeline.
- **G13 Engagement Analytics Dashboards — Taste-DNA Visualization & Cuisine Stats** — the preference viz + tracker stats.

Tech & dependencies

- **Tech:** Custom chart rendering (no chart libs) for the Food Taste DNA visualization (affinity per cuisine), the Cuisine Tracker counts + badge-milestone progress, and leaderboards, over a Firebase event/data pipeline.
- **Prerequisites:** M1G8 (Firebase data), M1G9 (Taste DNA).
- **Feeds into:** The profile/badge showcase; the "visualization of user's preferences" feature.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 0 Art (8) — **as proposed**, and workable: a social/utility app's art surface is **light-to-moderate** (UI, the Taste-DNA visual language, the badge/icon set, profile styling), which the **D11 conditional** can carry — though the **badge set** is a real, reusable visual-asset job worth scoping deliberately. That said, a **solo designer + 0 artists** is the case the design pool flags ("must ask the instructor which conditional bends", see [design-goals-pool.pdf](#)), and the design load here is heavy — the group-decision UX, the Community-Hub social design, the recommendation/onboarding and the moderation/safety UX — so a **2nd designer is recommended** (one owning the social/decision UX, the other the D11 visual surface + moderation UX). *Scope note (advisory):* the standout risk is **networking breadth** — a TCP chat relay is achievable, but a **custom reliable-UDP sliding-window media stream** (video/audio) is a research-grade task; make **text chat + images the floor** and treat live video/audio as a T2 stretch. Likewise, keep the **Bracket + Taste DNA + Firebase tracker** as the core M1/M2 spine, and let media streaming + advanced recommendations flex.

Track goals — suggested Design / UI-UX / TEAM picks for this project (no Art track as proposed — 0 artists, so the light Art surface + badge set is carried by the Design-owned D11 conditional); deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the group-decision USP, the Food Taste DNA, the Community Hub and the Cuisine Tracker/badges; the problem — friend groups can't agree where to eat; a primary benchmark e.g. Beli / Untappd, and a Singapore benchmark — e.g. a local food-discovery app)
- M1D2 Core-Loop Interactive Prototype (a clickable prototype of the swipe-bracket → recommendation and the hub/tracker flows, proving the UX before the engine is deep) · Figma/Unity
- M1D11 Art Proof of Concept *(conditional — mandatory for 0-artist teams; Design-owned)* — the visual identity: UI system, the Taste-DNA visual language, the badge/icon set and profile styling; 2 mockups.

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D11 Art Implementation Quality *(conditional — mandatory for 0-artist teams; Design-owned)* — the UI, Taste-DNA viz, badge set and profile implemented in-engine, no placeholders.

- **UI/UX M1:**

- M1U1 ★ UI Standards & Wireframe Baseline — standards for the swipe-bracket cards, the Taste-DNA viz, the Community Hub (communities/chat) and the profile/badges, plus a wireframe baseline; group-decision + social flows are the priority.

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — the bracket, hub/chat, tracker/badges and profile navigable unaided on the Native-Android target.
- M2U2 ★ UX Validation & Final Polished UI — validated with real friend groups (does the group swipe-to-decide actually resolve arguments?), polished on the Pixel 9a.

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial audio from the week-5 needs list present & working (swipe/pick cues, match-found, badge-earned, notification), suitable and correctly triggered — modest for a social app but the badge/match moments are real beats.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable playback.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — the app's audio set (swipe/pick + match + badge + chat/notification cues), suited to a mobile social app.
 - M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the playback the build needs (multi-channel; a distinct badge-earned + match cue).
 - M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the admin/moderation + restaurant/badge tool), prototype demo, team post-mortem (wk14)
-

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-22.*